Creative Software Design
6. Lab 2
Constructors & Destructors, `this`

Dept. of Computer Science

Hanyang University

Constructors and Destructors

- Both are special types of member functions
  - Constructors MUST have the same name as the class
  - Destructor MUST have the same name as the class with a prefix ~
  - Both have no return type
- Constructors can be overloaded
  - Constructors with no argument are default constructors
- Both are automatically called on allocation/deallocation
  - Even though they do nothing useful
- If no other constructor/destructor is defined, the compiler will create it

`this` Pointer

- Every instance has its own address allocated in memory
  - Every instance has different data
- Member functions are unique for the same class
  - All instances of the same class share member functions
- How can we access different members in the same member function?
  - A hidden argument is passed to the function
  - The argument `this` refers to the instance itself

#1 – Constructors and Destructor - 1

- Reuse example03.cpp in the previous session

```
$ cp ../week06_1/mat.* ./
```

- Add a set of constructors for given source code
  - Allocates `elements` as `new` operator
- Determine which constructor will be called

```
#include <iostream>
#include "mat.h"

int main(void) {
  int values[] = {1, 2, 3, 4, 5, 6};
  Matrix m1, m2(3,2), m3(values, 3, 2);

  m1.SetElements(values, 3, 2);
  m2.SetElements(values, 3, 2);

  m1.PrintElements();
  m2.PrintElements();
  m3.PrintElements();
  return 0;
}
```

#2 – Constructor and Destructor - 2

- Define a class of dynamic array for integers
  - Initailized with the given capacity
  - Each element should be stored in contiguous memory
  - The array automatically doubles in size when it reaches maximum capacity

```
#include <iostream>
#include "darray.h"

int main(void) {
  int i;
  DynamicArray darr = DynamicArray(5);
  for (i = 0; i < 10; i++) {
    darr.append(i);
  }
  for (i = 0; i < 10; i++) {
    std::cout << "arr[" << i << "]: " << darr.at(i) << std::endl;
  }
  return 0;
}
```


Constructors with Member Initializer Lists

- Another way to assign value by **member initializer list**
  - Primitive type members directly assigned the values
  - Non-primitive type members may induce different constructors

```
class Stock
{
public:
    string company;
    long shares;
    double share_val;
public:
    Stock(const string& co, long n, double pr)
    : company(co), shares(n), share_val(pr)
    {}
};
```

#3 – Member Initializer Lists

- Reuse example02.cpp in the previous session

```
$ cp ../week06_1/student.* ./
$ cp ../week06_1/data_mask.* ./
```

- Prevent ID from updating by removing `SetID()`
- Any student object with an invalid ID has
  - "9999999999" as `ID`
  - "No Name" as `name`
  - 0.0 as `GPA`

All Done!!

- Let's submit to ***gitlab***
- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure **pwd** at "week06_2")

```
# You are at {Your local project directory}/week06_2
$ git add *.h *.cpp Makefile
$ git commit –m "submission of 6-2"
$ git push origin main
```